SessionBound: Database-Enforced Task-Bound Sessions for Enterprise AI Agents

Minmin Wu
China Telecom Global Limited
wuminmin@chinatelecomglobal.com

***Abstract*—Enterprise AI agents need flexible SQL for audit, compliance, and operational analysis, but raw database access is over-privileged and fixed SaaS screens are too rigid for temporary business tasks. SessionBound is a database-enforced task-session model that turns an approved task into a signed token carrying scope, safe views, denied fields, TTL, operation policy, budgets, and policy-version claims. SessionBoundDB binds that token to a short-lived PostgreSQL credential and enforces the boundary through safe-view resolution, read-only SQL policy, disclosure accounting, aggregate/window shape gating, drift checks, and hash-chained receipts.**

We implement an accounting-complete wrapper path and a native PostgreSQL hook/executor hardening path. Evaluation covers API AST validation, a 140-case adversarial SQL suite, hook/executor checks, rollback-surviving receipts, credential-token drift, concurrency isolation, single-active binding, and path consistency. The adversarial suite blocks 130 boundary-violation cases while allowing and accounting for 10 safe-view detail-query cases; direct aggregate/window release is denied pending trusted templates. Compared with an RLS+safe-view+short-credential+audit baseline, SessionBound adds task-token binding, cumulative disclosure budgets, drift invalidation, and receipt chains. Full wrapper-path enforcement has 151.2–159.0 ms p50 latency on supported detail queries; structural guard checks are sub-millisecond, while unoptimized result accounting dominates larger detail-release stress settings.

***Index Terms*—AI agents, access control, database security, dependable computing, secure computing, authorization, SQL security, data governance, auditability.**

I. Introduction

AI agents change how enterprise users interact with data. A user no longer needs to know which report exists, which dashboard to open, or which API endpoint to call. Instead, the user can ask an agent:

> Analyze June 2026 travel reimbursements for the Sales department. Find unusual merchants, repeated claims, employee-level monthly totals, and claims near approval thresholds. Exclude salary and bank data.

This is exactly the kind of task agents are good at: temporary, exploratory, cross-table, and under-specified. It is also exactly the kind of task traditional enterprise software handles poorly. Building a permanent SaaS page or API for every low-frequency internal analysis task is expensive. Giving the agent raw database access is unsafe. Relying on prompts is not a security boundary. Relying on the application layer alone becomes fragile when SQL is generated dynamically.

Existing systems cover important pieces: delegated identity and OAuth scopes [1], [2], task-scoped tool authorization [3], data-product governance [4], [5], database-native policy enforcement [6], [7], [8], purpose-based database controls [9], [10], query auditing and inference control [11], [12], database proxies/reference monitors, and large-scale relationship authorization [13]. They do not, by themselves, turn one approved analytical task into a temporary SQL session with safe views, denied fields, budgets, and receipts. We therefore evaluate both a conventional RLS/safe-view/short-login baseline and a functionally equivalent composition that adds an external signature/credential PEP, tuple budgeting, and audit; the latter is equivalent only while every request traverses the external reference monitor. That is the gap SessionBound addresses inside the database boundary.

This paper argues for the following principle: business users approve tasks, not database policies. Agents generate SQL, but databases enforce the approved boundary. This paper frames the problem as a database-enforced authorization boundary for enterprise AI agents, rather than as an agent-planning or multi-agent coordination problem. SessionBound treats an approved enterprise task as the unit of database execution, rather than treating identity, role, relation, or individual query as the sole authorization unit. The agent can try. The database decides.

SessionBound separates a task control plane that approves and signs structured claims, an agent execution layer that proposes SQL, and a database runtime that deterministically enforces safe views, scope, denied fields, operation limits, budgets, and receipts.

This makes SessionBound a contract layer between enterprise approval and agentic database execution. The control plane records what the organization approved; the runtime enforces how that approval may be spent through SQL. This contract is not a database policy written by a DBA, nor a natural-language instruction interpreted by an LLM. It is a structured, signed, and accountable representation of an approved business task that both the agent runtime and the database can interpret deterministically.

SessionBoundDB is our PostgreSQL-based reference

runtime for SessionBound. It does not ask an LLM whether a query is safe. It does not depend on the agent correctly interpreting the task. It does not trust mutable session variables set by the agent. Instead, it validates a signed task token, exposes only registered safe views, rejects raw table access and denied fields, tracks query and disclosure budgets, and records receipts for evaluated allowed executions and SessionBound admission or release-barrier denials after binding.

The novelty is not any single primitive. It is the binding of task approval, credential identity, safe-view versioning, SQL surface, cumulative disclosure budget, and receipt semantics into one database-execution session object: enterprise task approval is compiled into an execution-time database session contract rather than scattered across separate credentials, views, policies, and audit logs.

A. Contributions

This paper makes five contributions.

1. Task-bound PostgreSQL session contracts. We instantiate TBAC/UCON-style task and usage state as a concrete session contract for untrusted, agent-generated SQL over approved database views.
2. Signed control-plane contract. Templates, applications, approvals, grants, TTLs, budgets, and task tokens become database-consumable structured claims rather than natural language.
3. Composed session object. Credential-token matching, safe-view versioning, database/view/dependency/option snapshot hashing, exposed-column hashing, disclosure budgets, aggregate/window shape policy, and receipts are bound into one session object.
4. PostgreSQL prototype with wrapper and native hardening paths. We implement an accounting-complete wrapper/API reference path and a native PostgreSQL hook/executor path for direct safe-view SQL, prepared-statement revalidation, `COPY (SELECT)`, utility checks, explicit cursor/`FETCH` and `EXPLAIN` denial, and executor-level accounting.
5. Evaluation against stronger baselines and attacks. We evaluate canonical behavior, adversarial SQL, credential-token binding, drift invalidation, a strong RLS+safe-view+short-credential audit baseline, ablations, scale behavior, and hook-only structural cost.

II. Motivation: Enterprise Tasks Are Approved Above the Database

SessionBound targets temporary enterprise analysis that is too specific for a permanent application screen and too sensitive for raw database access: reimbursement audits, vendor-concentration checks, compliance-log reviews, or operational investigations. These tasks need joins, aggregation, ranking, drill-down, and follow-up SQL, but the exact query plan is not known when the task is approved.

For example, a business user may request a June 2026 Sales travel reimbursement audit that excludes salary, phone, and bank-account fields, allows 20 SQL attempts and 5,000 result rows, and expires after 30 minutes. A manager or data owner can approve this task without writing RLS predicates or database grants. A data platform can expose safe views. A compliance policy can tighten budgets. The agent can then generate SQL inside the approved boundary, while the database rejects raw tables, denied fields, unsafe operations, and over-budget attempts.

III. Threat Model

SessionBound assumes the upper-layer agent is useful but not trusted. The agent may be confused, overconfident, prompt-injected, malicious, or simply wrong. The database boundary must remain meaningful even when the agent tries the wrong thing.

A. Trusted components

For this prototype, we trust:

- the task control plane that defines templates, evaluates grants, records approvals, and signs task tokens;
- the credential broker that issues short-lived database credentials;
- the database runtime that validates tokens and enforces policies;
- the PostgreSQL lock manager on one writable primary, including session-level advisory-lock cleanup when a backend exits;
- the cryptographic signing key or signing service used for task tokens;
- the integrity of registered safe views, runtime functions, and DB-local append-only audit storage.

In production, signing should use a KMS, JWKS, hardware-backed key service, or enterprise identity infrastructure. The prototype uses a simpler signing path. Receipt-chain tamper evidence assumes trusted DB-local append-only storage; DBA-resistant deployments should anchor chain heads to external WORM storage, SIEM, transparency logs, or enterprise audit infrastructure. The global single-active binding guarantee in this prototype is scoped to one non-split-brain PostgreSQL primary. Database-cluster split brain and cross-cluster global ownership require external primary fencing and are not solved by SessionBound's advisory-lock protocol.

The trusted-computing base (TCB) is therefore explicit rather than implicit: the signer and credential registry, the single-primary PostgreSQL lock and transaction manager, the `sessionbound_guard` hooks, the runtime SQL functions, the safe-view definitions and registry hashes, and the append-only receipt store. A safe-view definition is a policy artifact in this TCB; if it is over-broad, the runtime can only detect a hash/version drift, not infer that the definition is semantically unsafe. The database-enforced claim is limited to the reference-monitor interface below and is not a claim about arbitrary SQL or arbitrary semantic inference.

For the evaluated fragment, the reference monitor accepts one parsed single-statement `SELECT` (or the explicitly enumerated `COPY (SELECT)` utility form) whose relation

OIDs resolve to the token's approved safe-view registry. It rejects set-operation stacking, recursive CTEs, table functions, blocked payload aggregation, raw/catalog relations, mutations, DDL, and session-state utilities. The executor then stages and charges the complete result before release. For the common supported SELECT-F fragment, the wrapper and native paths implement the same conservative relation, function, denied-column, aggregate-shape, and release-barrier budget/receipt policy. The native path also covers `COPY (SELECT)` and fails closed on cursor/`FETCH` and `EXPLAIN` output until those utility surfaces have a release-barrier design. This is the scope of the reference-monitor argument, not a universal PostgreSQL equivalence theorem.

B. Untrusted or partially trusted components

We do not trust:

- the agent's prompt;
- the agent's reasoning;
- the agent's SQL generation;
- the agent's natural-language interpretation of the task;
- user-provided or database-provided text that may contain prompt injection;
- ordinary client-controlled session variables;
- application-layer filtering as the final boundary.

This is the main difference from designs that require an LLM to infer the precise intended operation from natural language. SessionBound does not ask the database to understand the user's natural-language intent. The database consumes structured claims from a signed token.

C. In-scope threats

SessionBoundDB is designed to mitigate:

- Sensitive field access: attempts to read salary, bank account, phone, identity number, credentials, or private notes.
- Raw table access: attempts to bypass safe views and query application schemas directly.
- Scope escape: attempts to read other tenants, months, departments, projects, or users.
- Unauthorized mutations: write, DDL, or other high-impact operations outside controlled commands.
- Prompt injection effects: data or user text instructs the agent to ignore policy or reveal secrets.
- Budget evasion: repeated queries, pagination, or alternate SQL forms attempt to disclose more detail rows than the task allows.
- Credential-token mismatch: attempts to combine one runtime credential with another task token.

D. Out of scope

The prototype does not solve:

- malicious DBAs, storage administrators, or compromised database hosts;
- compromised signing or audit infrastructure;
- side-channel attacks;

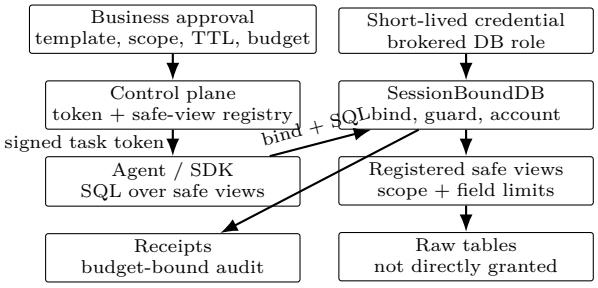

Fig. 1. SessionBound architecture. The approved enterprise task becomes a signed database-session contract; agents can attempt SQL, but the database runtime enforces the bound task state.

- complete formal verification of SQL equivalence;
- a production-grade formal SQL safety proof across all PostgreSQL features;
- cross-database federation;
- full differential privacy for statistical releases;
- arbitrary semantic inference across all allowed query answers.

SessionBound reduces and accounts for what an agent can discover. It does not claim to eliminate all inference and does not provide differential privacy.

IV. SessionBound Model

SessionBound has two major layers: a task control plane and a database runtime. The control plane turns enterprise task approval into a signed task token; the database runtime binds that token to a short-lived credential and enforces safe views, denied fields, budgets, aggregate policy, drift checks, and receipts inside the database session.

Task-bound session state machine. SessionBound treats an approved enterprise task as a stateful database authorization object. We write the session state as $S = \langle A, T, C, G, V, L, B, R \rangle$, where A is the approved task, T is the signed token, C is the brokered credential, G is global active-binding ownership, V is the safe-view registry snapshot, L is the permitted SQL surface, B is the remaining budget vector, and R is the receipt chain. Transitions include issue (approval to token), bind (token and credential to active database state), attempt (SQL shape and safe-view resolution), release (budgeted allowed result), deny (receipt-bearing rejection for decisions that reach the reference monitor), and expire/revoke/drift (fail-closed invalidation). PostgreSQL parser/analyzer failures that occur before a bound task decision are fail-closed observations rather than receipt-chain transitions. The contribution is this state machine around a database session, not a new identity credential, view mechanism, or audit log in isolation.

A. Template, application, approval, and token

A task template is defined by a data platform or application team, not by the agent. It names the task type, purpose, allowed safe views, denied columns, required scope fields, allowed operations, default TTL, budget vector,

and aggregate policy. A task application instantiates that template with a requester, actor, concrete scope, and business reason. A task approval decides whether that scoped task should exist; it may be automatic, role-based, or manual, but it does not require the approver to write SQL policies.

The approved task becomes a signed token consumed by the database runtime. In the prototype, the token binds the task id and type, delegator, actor, credential id, tenant and row scope, allowed views, denied columns, permitted operations, query and disclosure budgets, aggregate/window shape policy, safe-view registry version, policy version, database and view OIDs, view dependency and option hashes, view-definition and exposed-column hashes, audience, expiry, and nonce. The token is structured authorization data, not a natural language prompt.

B. Budgeted database session

A budgeted database session is a database session bound to one active task token. The session may issue open-ended SQL, but every attempt is checked against the token, safe view registry, operation policy, and remaining budget.

In the current prototype, a direct database connection may issue native `SELECT` over approved safe views after binding a signed task token. The PostgreSQL extension checks SQL shape and safe-view OIDs, then executor accounting charges every release-barrier-authorized output tuple (the legacy `unique_expense_rows` column is a conservative tuple counter) before tuples are forwarded. Projection aliases, joins, and non-aggregate detail CTE output cannot avoid the output-row charge; hidden source entities are not claimed to be counted. Direct aggregate/window release is denied pending approved templates. The same path covers SDK `query(sql)`, bound direct `SELECT`, prepared statements that are revalidated at `EXECUTE`, and `COPY (SELECT)`. Cursor/`FETCH` and `EXPLAIN` output are denied in this hardened revision. Raw table `SELECT` remains ungranted; schema resolution is allowed only so the hook can fail closed before release and emit receipts for denials that reach the bound reference monitor. Pooled connections can rebind only with a fresh valid token; prepared statements are dropped on unbind before the connection returns to the pool.

V. Control Plane: Templates, Applications, Approvals, and Budgets

The control plane is the enterprise layer that converts business approval into database-enforceable structure. A data owner approves a task such as "Alice may run a June 2026 Sales travel reimbursement review, excluding salary, bank-account, and phone fields, under a 30-minute TTL and bounded disclosure budget." The approver should not need to author `CREATE POLICY`, `ALTER ROLE`, or `GRANT` statements.

Roles grant eligibility to request task templates, not broad database access. For example, a finance reviewer may request finance-review tasks and a department manager may request department-scoped analysis tasks, but the database ultimately sees a concrete signed task token: actor X may analyze June 2026 Sales reimbursements through approved views under budget Y. Budget assignment remains a control-plane decision based on role, data classification, task purpose, and approval route; the database runtime enforces the resulting budget. In the artifact, control-plane issuance and administration endpoints require an issuer/admin API-key boundary carried in `X-TaskBound-Control-Plane-Key`. This keeps ordinary runtime callers from minting short-lived database credentials, issuing arbitrary task tokens, changing task templates or grants, or disabling receipt and budget-accounting options through request bodies.

VI. SessionBoundDB Runtime

SessionBoundDB is the PostgreSQL-based database runtime for SessionBound.

A. Session binding and credential-token binding

The runtime begins with two objects:

1. a short-lived database credential issued by the credential broker;
2. a signed task token issued by the control plane.

The credential authenticates the runtime connection. The task token authorizes the work.

At `bind_task`, the runtime checks:

- the task token signature is valid;
- the task token is unexpired and not revoked;
- the credential is unexpired and not revoked;
- the token `credential_id` matches the current session credential;
- the token `actor` matches the credential actor;
- the session login role matches the broker-issued database user;
- the token audience is `sessionbounddb`;
- this backend has no other active binding;
- no other PostgreSQL backend owns the same (`task_id`, `token_digest`, `credential_id`) binding key.

The last check is global within one writable PostgreSQL primary. `bind_task` derives a stable 64-bit session advisory-lock key from the exact binding key using a fixed SHA-256 prefix, obtains it with non-blocking `pg_try_advisory_lock`, claims a protected active row, allocates a fresh `binding_id` and monotonic `fence_token`, and only then installs backend-local trusted state. The active row is protected by uniqueness on (`task_id`, `token_digest`, `credential_id`) and on `binding_id`; budget, receipt, cleanup, and unbind mutations carry the current `binding_id`/`fence_token`. Hash collisions in the advisory-lock key can cause conservative rejection but cannot grant two owners the same exclusive lock.

A production implementation should make a stolen credential without a matching token insufficient, and a

stolen token without the matching credential insufficient. Credential-token binding is intended to reduce cross-task credential misuse.

PostgreSQL nuance: inside `SECURITY DEFINER` functions, `current_user` may refer to the function owner. Therefore, a production implementation should use `session_user` and/or an explicit credential registry mapping rather than relying on `current_user` alone. The measured prototype validates token signatures, token expiration, revoked task state, credential-id-to-token matching, actor matching, audience validation, cross-credential token replay rejection, and same-session rebind rejection in the tested prototype path; Section XIII reports those tests explicitly. Production deployments still require hardened credential brokerage, key management, and audit-retention integration outside this demo.

B. Safe views as the query surface

Agents do not query raw application tables. They query registered safe views. Safe views expose business objects, not internal schemas. This is complementary to database-native mechanisms such as PostgreSQL row-level security policies, which provide row filtering at the database layer [6].

Safe views are trusted policy artifacts reviewed by data platform, DBA, and security teams, not agents or ordinary business users. Agents should not have catalog privileges exposing sensitive view definitions. Migrations require review, hash/version updates, and task reapproval; a leaky safe view is a governance failure outside the runtime SQL guard.

Example safe views:

```
expenses
employees
departments
approval_events
ledger_entries
```

A safe view may:

- join raw tables into business objects;
- remove sensitive fields;
- apply tenant, department, month, or project scope;
- expose workflow hints;
- include stable business identifiers for budget accounting.

Safe views are not a complete inference-control solution. They reduce the discoverable surface and make it governable.

C. Operation policy

Native PostgreSQL `SELECT` is the database accounting entrypoint for read-only analytical SQL. The agent-facing SDK exposes this as `query(sql)`: the agent supplies ordinary SQL over approved safe-view names, and PostgreSQL hook and executor paths enforce the task boundary, debit query and disclosure budgets, and emit receipts for covered allowed releases and SessionBound denials.

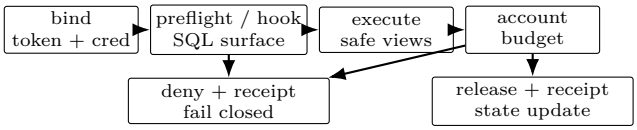

Fig. 2. Bind/query decision pipeline. Both paths stage and account the candidate result before release; the native path uses a private tuplestore so an over-budget query releases no prefix. Denials issued by the SessionBound guard or release barrier emit receipts; native parser/analyzer failures before these hooks are fail-closed observations rather than receipt-chain decisions.

`taskbound.run(sql)` remains as a compatibility wrapper. High-value writes should use controlled commands or stored procedures. The runtime rejects mutations, DDL, unsafe utility commands, raw schema access, denied fields, and blocked schemas.

D. Query decision pipeline

At runtime, every bound SQL attempt that reaches SessionBound admission follows the deterministic decision procedure in Algorithm 1. PostgreSQL syntax and name resolution failures that occur before the extension can associate a reference-monitor decision with the bound task fail closed outside this transition.

Algorithm 1. SQL decision pipeline for SessionBoundDB

Input: SQL attempt and bound database session state.

Output: Allowed result with receipt, or covered denial receipt.

1) Require ownership of the global active binding and validate `binding_id/fence_token`.
2) Validate the signed task token and its TTL.
3) Resolve all referenced relations against registered safe views.
4) Reject denied fields and blocked schemas.
5) Enforce row scope predicates for the bound task.
6) Reject disallowed operation types, including mutations, DDL, and unsafe utility commands.
7) Reserve query budget before execution.
8) Execute only after the structural checks pass.
9) Account every candidate output tuple before wrapper release or before native forwarding; aliases and aggregates use the same conservative output-tuple charge.
10) Emit an allow receipt for a covered release; otherwise reject the attempt and emit a denial receipt for denials produced by the SessionBound admission or release barrier.

The runtime does not ask an LLM whether a query is safe.

VII. Security Invariants

The SessionBoundDB enforcement contract can be described by a state

$$S = \langle A, T, C, G, V, L, B, R \rangle, \tag{1}$$

where A is the approved task, T is the signed task token, C is the credential/session binding, G is global active-binding ownership over (`task_id`, `token_digest`, `credential_id`) including `binding_id`, `fence_token`, advisory lock, and backend owner metadata, V is the safe-view registry and policy version, L is the permitted SQL surface, B is the remaining budget vector, and R is the receipt chain. Each covered SQL attempt that reaches SessionBound admission is evaluated by:

$$\begin{aligned} decide(q, S) \rightarrow\ & allow(result, receipt, S') \\ & \text{or } deny(reason, receipt, S'). \end{aligned} \tag{2}$$

These invariants define the enforcement contract implemented by the prototype and used to structure the adversarial evaluation. They are not a formal proof that all semantic inference is impossible.

- I1. Runtime credentials cannot execute arbitrary agent SQL or release task data without an active task binding; before binding, only public SessionBound runtime entrypoints such as `bind_task` may run.
- I1a. For one exact binding key, at most one PostgreSQL backend may be ACTIVE at any time on the same writable primary.
- I2. The bound token must match the session credential, actor, audience, expiry, token digest, and revocation state.
- I3. Referenced relations must belong to the approved safe-view registry.
- I4. Direct access to denied fields, raw schemas, catalog escape, mutation, DDL, and blocked payload aggregation is denied.
- I5. Scope predicates or session-bound claims constrain visible rows.
- I6. Budget state is monotonic: an allowed query consumes budget, or the query is denied; all budget and receipt mutations are fenced by the current `binding_id`/`fence_token`.
- I7. Every evaluated allowed execution and every SessionBound admission or release-barrier denial after binding emits a receipt; connection-level failures and native parser/analyzer failures before a task decision may be logged outside the task receipt chain.
- I8. Safe-view registry, policy version, or definition-hash drift invalidates the token or requires reapproval.
- I9. Direct aggregate and window release is denied unless an approved aggregate template supplies trusted provenance/cardinality logic.

In the prototype, I1–I2 and I8 are checked during `taskbound.bind_task`; I3–I4 are checked by API-layer AST preflight, the experimental `sessionbound_guard` PostgreSQL hook path, and database permissions; I5 is enforced by safe-view predicates; I6 is implemented through task execution state; I7 is implemented through execution receipts and covered denial receipts after binding; and I9 is implemented by conservative API, wrapper, and native shape checks: ungrouped aggregates, ad-hoc `GROUP BY`, `HAVING`, filtered aggregate release, and window-function release are denied unless a future approved aggregate template supplies trusted provenance/cardinality logic. Production implementations should harden these structural checks into always-on planner or executor integration, but the invariant set is independent of that implementation choice.

A. Security Guarantees for the Prototype SQL Fragment

We state the prototype security contract over a deliberately restricted SQL fragment. Let SELECT-F denote single-statement, read-only SQL consisting of `SELECT`, projection, predicates, joins, ordering and limits, and non-recursive CTEs over registered safe-view names. The fragment excludes direct aggregate release unless an approved aggregate template is used, ad-hoc `GROUP BY`, `HAVING`, filtered aggregate release, window functions, stacked statements, DDL, DML, `COPY`, `DO`, `CALL`, `CREATE FUNCTION`, temporary object creation, recursive CTEs, set-operation stacking, table sampling, table functions, raw-schema or catalog references, and high-risk payload aggregation such as `json_agg`, `jsonb_agg`, `array_agg`, `string_agg`, `xmlagg`, `row_to_json`, `json_build_object`, and `jsonb_build_object`. The evaluated `COPY (SELECT)` support is an engineering extension routed through the same relation-resolution and executor-accounting path; the propositions below are stated only for SELECT-F.

Let $\mathrm{Rels}(q)$ be the set of relation identifiers in q after database name resolution and before safe-view expansion. Let $\mathrm{Views}(T, V)$ be the approved safe views carried by token T and validated against registry V. Let $\mathrm{Cols}_{out}(q)$ be the direct column references that appear in the output expressions of q. A state $S = \langle A, T, C, G, V, L, B, R \rangle$ is well formed when the approved task A, token T, credential C, active binding ownership G, safe-view registry V, and permitted SQL surface L agree, the token is valid and bound to C, the safe-view registry and policy hashes match the token, the budget vector B is nonnegative, and the receipt chain R verifies. Under these assumptions, the prototype provides the following direct-reference and accounting guarantees. These are not guarantees against arbitrary semantic inference from allowed answers, and they are not differential privacy.

Proposition 1 (Safe-surface confinement). For any well-formed state S and query $q \in$ SELECT-F, if $decide(q, S) = allow(result, S')$, then $\mathrm{Rels}(q) \subseteq \mathrm{Views}(T, V)$. Consequently, every raw base relation evaluated during query execution is reached only through the definition of a registered safe view approved for the task; the query has no direct reference path to raw tables or catalogs.

Argument. The decision pipeline resolves relation identifiers and checks them against the task's approved safe-view registry before any result is released. Database permissions

deny raw-schema access, while the AST preflight and hook path reject resolved relation OIDs outside the approved safe-view set. Therefore any query that names an unregistered relation is denied. Raw base tables may still be read by trusted safe-view definitions, but not by direct agent-supplied SQL.

Proposition 2 (Denied-field non-disclosure for direct references). Let $D(T)$ be the denied-field set in token T, and let $\mathrm{Cols}(v, V)$ be the registered column list for safe view v. If $decide(q, S) = allow(result, S')$, then every direct output column reference $c \in \mathrm{Cols}_{out}(q)$ is in $\bigcup_{v \in \mathrm{Views}(T,V)} \mathrm{Cols}(v, V)$ and $c \notin D(T)$. Thus a column that is absent from the approved safe-view column lists, or that appears in the denied-field set, cannot be directly projected by an allowed query.

Argument. In SELECT-F, output columns must resolve against the relations that survived Proposition 1. A column not exported by an approved safe view cannot be bound by name. At bind time, the database rejects a task token if any approved safe view still exposes a token-denied column. At query time, column references and output aliases that match the normalized denied-field set are rejected by the structural policy before execution. The guarantee is syntactic and direct: it does not rule out inferences about denied fields from permitted columns or aggregates.

Proposition 3 (Monotonic budget accounting). Let budgets be ordered componentwise by $\preceq$, and let Δ be the nonnegative budget charge instantiated by the prototype semantics in Table II. For any allowed query, $decide(q, S) = allow(result, S')$ implies $0 \preceq B' \preceq B$, where B' is the budget component of S'. If a candidate result is not covered by the remaining conservative output-tuple budget, both paths deny the query before releasing any tuple. The native path stages the complete result in a private tuplestore, performs fenced accounting and receipt emission, and only then hands the stored tuples to the client. The prototype therefore does not claim an `expense_id`-derived unique-entity budget: projection aliases, joins, and non-aggregate detail CTE output are charged by emitted tuple count, while direct aggregate/window release is denied pending approved templates and semantic leakage from hidden source entities remains outside this guarantee.

Argument. The runtime treats budget dimensions as nonnegative counters. After structural admission, both paths reserve the execution attempt through a fenced query-count transition before running the statement. They then stage the candidate result, check aggregate-shape and tuple-budget policy, and execute one fenced release-barrier transition that debits released output tuples and appends the decision receipt before release. Both paths subtract only nonnegative charges and record the resulting state in a receipt. Therefore the remaining budget cannot increase across an allowed transition; an over-budget transition reaches a denial state before any candidate tuple is released.

Proposition 4 (Receipt-chain tamper evidence under trusted DB assumptions). Assume collision-resistant hashing and append-only receipt storage inside the trusted database boundary. For a receipt chain $R = (r_1, \ldots, r_n)$ where each r_i stores the hash of r_{i-1} and a hash of its own canonical contents, removing or modifying any interior receipt r_j, $1 < j < n$, changes r_j's hash or breaks the previous-hash value stored in r_{j+1}. The change is detected by chain verification unless the adversary can violate append-only storage or find a hash collision.

Argument. Each receipt commits to the previous receipt hash and to a typed JSONB material containing the receipt id, execution id, sequence, binding fence, current decision, query hash, timestamp, touched views, actor, reason, and budget delta. Changing an interior receipt changes its hash; deleting it changes the predecessor expected by the next receipt. Under append-only storage, the adversary cannot rewrite the downstream chain to hide the mismatch. This is a tamper-evidence property for the audit trail under trusted DB assumptions, not a Byzantine storage guarantee against malicious DBAs, storage administrators, compromised hosts, or compromised signing/audit infrastructure.

Proposition 5 (Global single-active binding under a trusted PostgreSQL lock manager). Assume one writable PostgreSQL primary with a correct session-level advisory lock manager. For one exact (`task_id`, `token_digest`, `credential_id`) binding key, at most one PostgreSQL backend can be ACTIVE at a time.

Argument. The same binding key maps deterministically to the same 64-bit session advisory-lock key. PostgreSQL grants an exclusive session advisory lock for that key to at most one lock-holding backend at a time. The SessionBound backend cannot enter ACTIVE until it has acquired the lock, claimed the exact protected active row, and installed the resulting `binding_id`/`fence_token` in trusted backend-local state. Query execution validates that local state against the active row and the still-held advisory lock. If an owner disconnects or is terminated, PostgreSQL releases the session lock; a new backend may recover the stale row only after acquiring the lock and verifying that the recorded old owner no longer appears with the same PID, backend start, postmaster start, and database OID. Budget, receipt, cleanup, and unbind mutations are fenced by `binding_id`/`fence_token`, so delayed operations from an old owner cannot mutate a replacement owner. A 64-bit lock-key hash collision can conservatively reject an otherwise independent binding, but it does not permit two simultaneous owners of the same advisory lock. This is not a distributed consensus claim and does not address PostgreSQL HA split brain.

VIII. Anti-Evasion Scope

SessionBound handles direct boundary violations and accounts for bounded disclosure; arbitrary semantic inference remains outside its formal guarantees.

A. Direct boundary violations

SessionBoundDB can directly block:

- access to raw schemas;

- access to sensitive fields that are not exposed through safe views;
- queries outside row scope;
- unsupported operation types;
- repeated querying beyond task budgets;
- use of credentials without a matching task token;
- attempts to continue after token expiry or revocation.

B. Inference and side channels

SessionBound does not claim to solve arbitrary inference from allowed answers. For example, if a safe view exposes enough correlated information, an agent may infer sensitive facts indirectly. The prototype now denies direct aggregate/window release pending approved templates, but arbitrary semantic inference across multiple allowed detail answers remains outside its formal guarantees. If future aggregate templates re-allow statistical release, they must handle differencing attacks such as subtracting overlapping group sums to infer one entity's contribution. Future mitigations include predicate-family budgets, aggregate templates, differencing detection, entity-column budgets, repeated-probe penalties, and optional DP for statistical release; these are not implemented here. Timing and resource side channels are also outside the prototype's formal guarantees. Classical database inference-control work studies these risks for statistical databases and repeated query answers [14].

C. Adversarial SQL patterns

Table I summarizes representative evasion patterns, defenses, and limitations.

This table defines the prototype's anti-evasion scope and the path toward production hardening.

IX. Budget Semantics

SessionBound disclosure budgets limit how much task-authorized information an agent may retrieve during one approved analytical session. They are authority accounting, not differential-privacy accounting [15].

A production deployment should use a vector rather than a single unique-row limit. Useful dimensions include query count, result rows, result bytes, per-query payload bytes, aggregate payload shape, unique business entities, column weights, detail-versus-aggregate multipliers, and small-group penalties.

The prototype uses a conservative output-tuple counter (the legacy column name `unique_expense_rows` is retained for schema compatibility), while the broader model treats budget as a vector of exposure counters tied to task authority. The current runtime also carries a configurable `min_group_size` policy in the signed task token. Direct aggregate release is denied by default until a task-specific aggregate template can provide trusted source-entity provenance and cardinality. This includes ungrouped aggregates, ad-hoc grouped aggregates, `HAVING` probes, filtered aggregate release, and window release. Covered aggregate/window policy denials emit denial receipts and do not release disclosure rows.

Table II gives the exact prototype charge semantics used for Δ. It is intentionally operational: it counts candidate output tuples, not information theoretic leakage.

Budget vectors are not mutual-information bounds. They provide operational authority accounting, auditability, and a governance lever for approved enterprise tasks. High-sensitivity deployments can charge entity-column pairs or add repeated-probe penalties.

X. Schema Drift and Safe View Versioning

Task tokens should not silently survive changes to the data boundary they approve. If a safe view changes after approval, the token may no longer represent the approved task.

A task token should therefore bind safe views by more than name: registry version, policy version, database object identifier, view OID, view dependency hash, view option hash, view definition hash, and exposed-column hash. PostgreSQL views make this important: `CREATE OR REPLACE VIEW` may preserve `pg_class.oid` while changing the definition, and `relfilenode` is not an appropriate view identity. The prototype binds registry, policy, database/view identity, dependency, option, definition, and exposed-column hashes into the signed token and fails closed if the runtime snapshot no longer matches the approved boundary.

XI. Receipts

SessionBound receipts are structured audit records that bind task identity, query decision, and budget impact. They are not merely logs.

Each receipt records a receipt id, execution id, task id, actor, query hash or command name, decision, rows returned, budget delta, remaining budget, timestamp, previous receipt hash, and current receipt hash. The schema also includes a touched-safe-view list. Query receipts populate this list from PostgreSQL parse/analyze safe-view OIDs for safely parseable bound database SQL and then map those OIDs back to approved registry names. API preflight receipt writes and wrapper-side unsupported syntax rejected before database execution use an approved-name fallback. Native PostgreSQL parser/analyzer failures that occur before hook admission are fail-closed scoped observations rather than task-chain receipts. The touched-safe-view field is hashed audit context rather than the enforcement predicate; relation enforcement still comes from AST and safe-view OID checks.

The prototype implements a lightweight hash chain: each receipt stores `previous_receipt_hash` and `receipt_hash`, where the current hash commits to the receipt id, execution id, task id, budget account, binding id, fence token, query hash, decision, rows returned, budget delta, remaining budget, reason, actor, touched-view list, timestamp, and previous hash. This does not provide a full external proof system, but it detects

TABLE I
ADVERSARIAL SQL PATTERNS AND SESSIONBOUND DEFENSES.

Pattern	Example	Defense	Limitation
Raw table escape	`SELECT * FROM app_data.expenses`	no raw grants; safe views only	strong in prototype
Sensitive column	`SELECT salary FROM employees`	denied fields; safe view exclusion	direct leakage only
Scope escape	query another month or department	safe-view predicates from task claims	strong if views are correct
Mutation	`DELETE`, `UPDATE`, `INSERT`	operation policy; read-only `run` path	writes require controlled commands
DDL	`DROP`, `ALTER`, `CREATE`	blocked operations	production should use hook-backed validation
Catalog escape	`pg_catalog`, `information_schema`	blocked schemas and no grants	production needs planner/executor hooks
Pagination scraping	repeated `LIMIT/OFFSET`	query budget and disclosure budget	budget model matters
JSON / array payload aggregation	payload aggregation functions	conservative AST and hook denial; template allowlist is future work	high-risk payload functions are blocked rather than sensitivity-scored
Aggregate inference	small-group `GROUP BY`	direct aggregate, `GROUP BY`, `HAVING`, and window release are denied pending approved aggregate templates	arbitrary semantic inference remains out of scope
Timing side channel	expensive predicates	statement timeout and resource budget	partial

TABLE II
PROTOTYPE BUDGET-CHARGE SEMANTICS.

Budget dimension	Prototype charge
`query_count`	Wrapper and native paths: one debit in a fenced admission transition after structural admission and before execution. This charges execution attempts that later fail or are denied at release. Preflight and structural denials before admission emit denial receipts without result/entity debit in the current prototype.
`result_rows`	Number of staged output tuples authorized by the fenced release barrier and recorded in state/receipts. Join multiplicity is visible here, so duplicate output tuples increase this counter. The prototype's legacy `rows_returned` receipt field is not a proof that the network/client consumed every tuple after authorization.
`disclosure_tuples`	Every candidate result tuple consumes one unit, independent of projected names, aliases, joins, or whether an identifier column is present. This is conservative and does not claim to count hidden source entities in aggregate results.
`aggregate_group`	Direct aggregate release is denied unless a future approved aggregate template supplies trusted provenance/cardinality logic. This includes ungrouped aggregates, ad-hoc `GROUP BY`, `HAVING`, filtered aggregate release, and window-function release.
payload aggregation	High-risk payload aggregation functions such as `json_agg`, `array_agg`, and `string_agg` are denied unless a future template explicitly allows them.
over-budget native execution	The complete result is staged; if the tuple charge exceeds the remaining budget, no tuple is forwarded and an autonomous zero-release denial receipt is recorded.

post-hoc sequence changes within the trusted database boundary under append-only audit storage. It does not protect against malicious DBAs, compromised hosts, storage administrators, or compromised signing/audit infrastructure; DBA-resistant deployments should anchor chain heads to external WORM storage, SIEM, transparency logs, or enterprise audit infrastructure.

XII. PROTOTYPE IMPLEMENTATION AND PRODUCTION HARDENING

A. Prototype

The current prototype includes:

- FastAPI control plane;
- task template and grant registry;
- short-lived PostgreSQL credential broker;
- signed task token issuance;
- PostgreSQL runtime functions;
- safe views over travel reimbursement data;
- `taskbound.bind_task(payload, signature)`;
- an agent SDK with `TaskboundSession.query(sql)`;
- native guarded safe-view `SELECT` plus compatibility `taskbound.run(sql)`;
- sensitive field denial;
- blocked raw table access;
- query and disclosure budget state;
- receipts for evaluated allowed executions and SessionBound admission or release-barrier denial decisions;
- controlled commands for high-value workflow actions;
- an agent-agnostic HTTP evaluation harness.

B. Implementation honesty

The current hardening prototype uses API-layer AST preflight, PostgreSQL hooks, executor result accounting, rollback-surviving audit writes, PostgreSQL permissions, and a global single-active binding protocol. Agents do not receive raw table `SELECT` grants. The SDK's `query(sql)` executes native SQL over the bound safe-view schema; a direct `SELECT ... FROM taskbound.expenses` is allowed only after binding and is observed by executor accounting before the staged result is authorized for forwarding. The hook is not the sole DDL/DML protection: runtime credentials are deny-by-default and lack raw-table and DDL privileges, while `ProcessUtility_hook` fail-closes and logs utility statements that reach the runtime. The wrapper reference path instead materializes the candidate

TABLE III
IMPLEMENTATION AND EVALUATION PATHS. THE PATHS ARE DELIBERATELY SEPARATED SO THAT WRAPPER REFERENCE OVERHEAD, NATIVE HARDENING COVERAGE, AND HOOK-ONLY STRUCTURAL COST ARE NOT CONFLATED.

Path	Purpose	Enforcement and accounting	Used for	Limitation
Wrapper/API reference path	Compatibility and accounting-complete reference implementation	API AST preflight, database permissions, and `taskbound.run`; PL/pgSQL materialization accounts conservative output tuples; allow/deny receipts are emitted through the autonomous audit channel	Canonical validation, adversarial SQL API suite, default p50 overhead, and wrapper scale sweep	Not optimized; 10k-row stress bottleneck is materialization/accounting
Native PostgreSQL hook/executor path	Production hardening direction for direct safe-view SQL	`post_parse_analyze_hook`, `ProcessUtility_hook`, executor hooks, safe-view OID checks, aggregate/window shape gating, staged tuple accounting, and receipts	Direct DB enforcement suite, prepared-statement revalidation, cursor/`FETCH` and `EXPLAIN` denials, `COPY (SELECT)`, rollback audit, SDK smoke tests, and diagnostic native end-to-end benchmark	Current native accounting is measured but unoptimized; broad production workloads remain future work
Hook-only structural microbenchmark	Isolate parse/analyze structural guard cost	Structural SQL guard only; no row execution, no disclosure accounting, and no receipts	0.101–0.129 ms p50 structural-check cost across SELECT/JOIN/non-aggregate CTE detail shapes; GROUP BY/window denials pass	Not an end-to-end SessionBound performance claim

result, then checks group and conservative tuple-budget policy before returning rows. High-value writes go through controlled commands.

Binding lifetime is session-control state rather than ordinary agent transaction state. `bind_task` validates the signed token and credential, obtains a session-level advisory lock for the exact binding key, claims a protected active row, and installs trusted C-local binding state. The active row stores the token digest and signed nonce, credential id, `binding_id`, monotonic `fence_token`, advisory-lock key, lock backend, owner PID, owner backend start, postmaster start, database OID, session user, acquisition times, and token expiry. `unbind_task` releases only the caller's fenced row before releasing the advisory lock. Crash recovery first reacquires the same advisory lock, then recovers an old row only if the recorded owner is not live by PID, backend start, postmaster start, and database OID; `last_seen_at` is diagnostic and is not a takeover lease.

The database hook path is implemented by the `sessionbound_guard` C extension, loaded through `shared_preload_libraries`. The extension registers `post_parse_analyze_hook`, `ProcessUtility_hook`, and executor hooks. Trusted SUSET GUCs populated by `taskbound.bind_task` carry the task, budgets, receipt flags, and approved safe-view OIDs. The executor path wraps the destination receiver and stages the complete result in a private tuplestore. It charges output tuples before authorizing the staged result for forwarding; no `expense_id`-only projection heuristic is used. For allowed releases, receipt and budget updates occur in one fenced release-barrier transition on the autonomous same-database audit channel; the receipt cardinality is the authorized staged-row count, not a client-delivery acknowledgment. Controlled-command business mutations and allow receipts execute on the same autonomous, fenced database channel so caller transaction rollback cannot erase the command effect while leaving a task decision ambiguous. API preflight, wrapper, hook, controlled-command, and release-barrier denials use the same audit channel for rollback-surviving denial receipts when the SessionBound runtime can associate the denial with a bound task. Native PostgreSQL parser/analyzer failures before that point fail closed but are not represented as task-chain receipts in the prototype.

The hook rejects non-`SELECT` statements, raw `app_data` relations, catalog access, recursive CTEs, `UNION/INTERSECT/EXCEPT`, unapproved relation OIDs, non-view relations, unsafe non-allowlisted functions, sensitive output aliases, and high-risk payload aggregation functions such as `json_agg`, `jsonb_agg`, `array_agg`, `string_agg`, `xmlagg`, `row_to_json`, `json_build_object`, and `jsonb_build_object`. The API AST validator remains defense in depth and provides portable preflight feedback before the database runtime is invoked. Aggregate/window release is gated conservatively across API, wrapper, and native paths: direct aggregate release is denied pending an approved aggregate-template mechanism with trusted provenance/cardinality logic. This includes ungrouped aggregates, ad-hoc `GROUP BY`, `HAVING`, filtered aggregate release, and window-function release.

This should still be read as an experimental hook path, not a production SQL-safety proof. A production implementation should harden the same structural policy into:

- always-on PostgreSQL parser/analyzer hooks;
- planner hooks;
- executor hooks;
- extension-level session state and registry management;
- deny-by-default schema grants;
- blocked unsafe utility commands;
- executor-level result accounting for deployments that allow native agent `SELECT` syntax, including query-

budget debit before execution, bounded result materialization or streaming interception, disclosure-budget accounting before client release, and receipt emission through an audit path that survives transaction aborts;
- `statement_timeout` and resource limits;
- cleanup for expired short-lived roles;
- denial logging outside user transactions.

The measured AST prototype extracts referenced relations, columns, function calls, CTEs, recursive CTEs, subqueries, joins, catalog access, operation type, and set operations. It blocks raw schema references, catalog access, mutations, DDL, `COPY`, `SET`, `SHOW`, `CREATE FUNCTION`, `DO` blocks, recursive CTEs, `UNION`/`INTERSECT`/`EXCEPT` stacking, unknown functions, denied-field aliases, and high-risk payload aggregation functions. The API AST validation evaluation passed 20 of 20 query-shape cases, and the latest PostgreSQL hook raw result passed 19 of 19 direct database enforcement cases; the current hook script also includes a follow-up prepared-plan rebind case for the newer static review.

C. Query example

Allowed:

```
SELECT expense_id, department_name, category, amount, status
FROM expenses
WHERE expense_month = '2026-06'
ORDER BY amount DESC
LIMIT 10;
```

Denied:

```
SELECT employee_name, salary FROM employees;
```

Denied:

```
SELECT * FROM app_data.expenses;
```

Denied:

```
DELETE FROM expenses WHERE expense_month = '2026-06';
```

XIII. Evaluation

The prototype evaluation has two goals:

1. validate security correctness and boundary behavior for approved and adversarial SQL;
2. diagnose performance and production feasibility across the wrapper reference path, stronger baselines, and native structural checks.

The evaluation focuses on functional boundary validation, security-property comparisons against narrower database controls, database-runtime overhead, and concurrency behavior on the default synthetic dataset.

A. Experimental setup

The reported evaluation used Docker Compose with PostgreSQL 16.14, the FastAPI SessionBoundDB prototype, and the `sessionbound_guard` extension loaded through `shared_preload_libraries`. The default seed contains 430 expenses, 240 employees, and 124 departments. The artifact metadata records the git commit identifier, environment variables, raw result paths, and run timestamps.

Experiments ran on Ubuntu 22.04.5 LTS under WSL2 on a 13th Gen Intel Core i9-13900HX host with 32 logical CPUs and 15 GiB RAM, using Docker 29.1.3, Docker Compose v2.40.3, and Docker Desktop `overlay2` storage. PostgreSQL used default settings except for `shared_preload_libraries`. API measurements used local Docker networking; database-only measurements used direct connections from the Compose network. The benchmark harness did not intentionally generate competing load.

The AST and adversarial SQL evaluations exercise the HTTP API, including dynamic credential issuance, signed task-token creation, token binding, AST preflight, the wrapper reference query path, budget state, and receipts. A separate SDK surface evaluation calls `TaskboundSession.query(sql)` directly from the Compose network. The hook evaluation bypasses the API query endpoint for agent cases by using generated runtime credentials directly against PostgreSQL, and it uses trusted superuser setup for hook-only structural checks. The overhead breakdown uses direct PostgreSQL connections from the Compose network so that it isolates database runtime overhead from HTTP, model calls, dynamic credential creation, and API-layer AST parsing. A hook-only microbenchmark measures `sessionbound_guard_check(sql)` directly, exercising PostgreSQL parse/analyze and the structural guard without executing result rows or performing wrapper materialization.

B. Artifact reproducibility

The artifact is container-oriented and includes code, synthetic data, raw outputs, and LaTeX source. A reviewer starts the database, extension, API, and seed data with `docker compose up -d --build`, then runs the scripts in `paper/tdsc/scripts/` against `localhost:8000` and PostgreSQL on `localhost:15432`. The reproduction entrypoints are `ast_validation_eval.py`, `adversarial_sql_eval.py`, `sessionbound_guard_hook_eval.py`, `overhead_breakdown.py`, `hook_microbenchmark.py`, `rollback_audit_eval.py`, and `native_end_to_end_benchmark.py`. A `path_consistency_eval.py` script compares the API endpoint, direct `taskbound.run(...)` wrapper, and direct native safe-view SQL over one 12-case corpus. A `scope_completeness_eval.py` script checks task-allowed safe views against raw-table scope provenance. A global `single_active_binding_eval.py` exercises same-key races, stale recovery, fencing, rollback semantics, and different-key concurrency. A targeted `native_partial_budget_eval.py` validates projection-, alias-, and aggregate-independent atomic tuple accounting; `concurrent_isolation_eval.py` validates task isolation. Exact command lines and environment variables are recorded in the artifact README and changelog.

The artifact manifest maps each table to its raw JSON/CSV result files. Scripts write timestamps, git commit, iteration counts, error counts, and measured latencies into those files; the tables below are derived from raw outputs rather than hand-entered experimental claims.

C. Functional validation

We evaluated the SessionBoundDB PostgreSQL prototype using Docker Compose. The artifact contains the evaluation scripts, validation reports, benchmark outputs, and source code needed to reproduce the results. The canonical validation suite passed 24 of 24 scenarios; the full scenario matrix and raw receipts are provided in the artifact repository rather than reproduced in the main text.

Scope violations over safe views are enforced by filtering rather than explicit denial. A query for another approved-view but out-of-scope month returns zero rows because the safe view predicate binds the session to the task scope. The scope-completeness run `scope_completeness_20260719_112307.json` passed 4 task scenarios and 14 non-vacuous view checks, matching every returned expense, dimension, workflow, and ledger row to raw-table tenant/month/department provenance. The measured prototype conservatively blocks payload aggregation functions such as `json_agg`, `array_agg`, `string_agg`, and `row_to_json`; direct ad-hoc aggregate and window release is also denied until an approved aggregate-template mechanism can supply trusted provenance and cardinality logic. Aggregate signals that are needed by a task should be encoded in reviewed safe views or future approved templates rather than released through arbitrary agent SQL.

D. Baseline security comparison

We compare SessionBound against four narrower database configurations to separate access-surface reduction from task-bound session enforcement:

- **Raw PostgreSQL**: direct access to application tables using a trusted analyst/admin test role.
- **Role-only**: a constrained database role with coarse grants but no task token, budget vector, receipt chain, or task-specific safe view binding.
- **Safe-view-only**: curated views and denied-field exclusion, without credential-token binding, query budgets, disclosure budgets, or signed task tokens.
- **RLS + Safe View + Short Credential + Audit**: PostgreSQL RLS policies over raw tables, field-limited safe views, a short-lived read-only role, and basic query-hash/actor/timestamp/row-count audit logging, but no task token, credential-token binding, disclosure budget, receipt hash chain, or safe-view token invalidation.
- **SessionBound full**: signed task token, short-lived credential binding, safe views, denied fields, operation policy, query budget, disclosure budget, aggregate/window shape gating, anti-payload aggregation checks, and receipts.
- **Functionally equivalent external PEP composition**: the RLS/safe-view role, short-lived login, an external signature/credential PEP, Python-side cumulative query/tuple budgeting, persistent PEP state, execution IDs, and chained receipts. The artifact tests this composition separately and also tests a direct-role bypass, making its external PEP part of the stated TCB rather than silently treating the feature-removed baseline as a novelty proof.

For fairness, the safe-view-only baseline uses pre-scoped or manually parameterized views matching the evaluated task scope, but it lacks signed task binding, cumulative budgets, drift invalidation, and decision-bound receipts. The RLS+Safe View+Short Credential+Audit baseline is intentionally strong and may use pre-scoped or protected trusted-session predicates. Even then, it lacks signed task-token binding, credential-token matching, cumulative disclosure budget, safe-view drift invalidation, and decision-bound receipts. A client-writable RLS scope parameter is unsafe; a protected one becomes bespoke control-plane/runtime plumbing rather than plain RLS alone.

The functionally equivalent composition passes the same cumulative tuple-budget, query-budget, credential-replay, execution-id idempotence, and chained-receipt checks for the comparison workload when every request traverses its external PEP. The same baseline role can nevertheless bypass that PEP on a direct database connection; this explicit bypass case is the reference monitor distinction measured by the artifact and is why the result is not presented as a proof that the primitives are individually novel.

Table VI summarizes the observed security properties. In the updated credential-token test suite, SessionBound denies credential-id mismatch, cross-credential token replay, wrong audience, wrong actor, expired tokens, revoked tasks, and same-session rebind to a second active task. It also rejects tokens whose safe-view registry version, policy version, database/view identity, view dependency hash, view option hash, view-definition hash, or exposed-column hash no longer matches the runtime registry. The comparison separates systems with different policy substrates and guarantees: Qapla rewrites queries with SQL-stored row/column policies, Blockaid checks web-application queries against data-access policies with SMT-backed verification, Sieve scales large fine-grained policy sets through middleware query planning, and PICACHV verifies data-use policy enforcement over relational-algebra plans. SessionBound's narrower claim is the composition of task-token, credential, safe-view snapshot, budget, and receipt state inside one PostgreSQL session.

E. Adversarial SQL evaluation

The current adversarial evaluation uses an SQL suite with 140 cases. The suite covers direct boundary violations, raw-schema escape, catalog and metadata escape, `pg_temp`/search-path/GUC tampering, function abuse, payload aggregation and compression, JSON/XML/composite leakage, prepared-statement and cursor lifecycles, `COPY` and `EXPLAIN` variants, CTEs, recursive CTEs, set operations, `VALUES`/`LATERAL`/`DISTINCT ON`/table sampling, DDL/DML/utility commands, aggregate-inference probes, pagination and budget scraping,

revocation/expiry/rebind attempts, schema drift, and rollback/audit-survival cases. The 2026-07-19 rerun passed all expected classifications: 130 cases were blocked and 10 safe-view detail-query cases were allowed and accounted for. Direct aggregate and window release cases are classified as blocked pending approved aggregate templates.

This result should not be read as a proof of complete SQL safety. It does show that the current prototype is no longer limited to literal string checks for the main tested attack families. The current safe aggregate policy is conservative: direct aggregate/window release is denied rather than treated as solved by caller-supplied group-size aliases. Arbitrary semantic inference across multiple allowed detail answers remains out of scope.

We separately smoke-tested the native SDK surface. A fresh 2026-07-19 Docker smoke test confirmed that bound direct `SELECT ... FROM taskbound.expenses` returns buffered detail rows only after the fenced release-barrier transition; scoped `employees`, `departments`, `approval_events`, and `ledger_entries` stayed within the signed `2026-06 + dep_sales` task scope; and native ungrouped aggregate, direct-entity grouping, filtered aggregate, window, `pg_sleep`, direct runtime-helper, and denied-column bind probes failed closed.

The 2026-07-20 database-hook raw result evaluates the `sessionbound_guard` path directly with 20 cases. It confirms GUC tamper resistance, allowed bound native `SELECT`, prepared `EXECUTE` under the current binding, prepared-plan cross-binding rebind denial, and `COPY (SELECT)`, and blocks set operations, catalog/raw access, runtime-helper abuse, payload aggregation, unapproved OIDs, `HAVING`/direct-entity aggregate shapes, direct aggregate/window release, cursor/`FETCH`, holdable cursors, and `EXPLAIN` output. The companion `prebind_runtime_20260720_001459.json` probe passed 7 of 7 cases, confirming that runtime credentials cannot run ordinary SQL, utility/TEMP SQL, side-effect functions, prepared statements, or private runtime helpers before binding, while `bind_task` remains callable.

A separate path-consistency run, `path_consistency_20260720_080527.json`, passed 14 of 14 cases across API, direct-wrapper, and direct-native paths. It compares 42 path observations for allowed detail projection, join, non-aggregate CTE, denied-column, raw-schema, catalog, set-operation, aggregate/window, side-effect-function, mixed runtime-entrypoint, `TABLESAMPLE`, and release-time lifecycle queries. Two direct-native PostgreSQL parser/analyzer rejections (`salary` absent from the hardened safe view, and `TABLESAMPLE` on a view) are recorded as scoped observations rather than receipt-equivalent task decisions.

The receipt-fault run `receipt_fault_20260720_000538.json` passed 8 of 8 cases. It recomputed receipt hashes, checked contiguous per-task receipt sequences and previous-hash links, verified tamper sensitivity, blocked direct agent receipt forgery and helper calls, and covered controlled-command allow/deny receipt insertion plus allowed-command rollback survival. The current receipt claim is therefore limited to these evaluated allowed executions and SessionBound admission or release-barrier denials, not to native parser/analyzer failures before the hook can associate a decision with the task chain.

The control-plane authentication run `control_plane_auth_20260719_140226.json` passed 8 of 8 cases. Missing or wrong keys were rejected for credential issuance, task issuance, and administrative registry reads; the configured key permitted the expected calls; and request-body attempts to disable receipts or budget accounting were rejected before token issuance.

F. Performance overhead and ablation

The latest overhead breakdown measures five query patterns: simple `SELECT`, `JOIN`, `GROUP BY`, non-aggregate CTE detail query, and window function. Each supported pattern uses 10 warmup iterations and 100 measured iterations per mode. The run compares raw PostgreSQL, role-only, safe-view-only, the RLS+safe-view+short-credential+audit baseline, SessionBound without receipts, SessionBound without budget updates, and full SessionBound. Table VII reports p50 latency in milliseconds. The SessionBound aggregate/window rows are marked unsupported because the current policy denies direct aggregate/window release pending approved templates; all measured rows completed with zero query errors.

Raw, role-only, and safe-view-only queries are sub-millisecond on the default seed. The stronger RLS+safe-view+short-credential+audit baseline adds a basic audit insert and measures 0.91–1.28 ms p50. Full SessionBound measures 151.2–159.0 ms p50 across the supported detail-query patterns after statement-start global-owner validation and fenced mutation checks. Disabling receipts or budget accounting does not consistently reduce latency, so receipt insertion and budget updates do not dominate this small-dataset benchmark. The remaining cost is primarily repeated session-control validation, wrapper runtime checks, and PL/pgSQL materialization.

1) Performance diagnosis: The database-runtime breakdown isolates direct PostgreSQL execution and therefore does not include HTTP latency, model calls, dynamic credential creation, or API-layer AST preflight. End-to-end agent requests do incur the AST preflight before entering PostgreSQL, so the end-to-end path is strictly larger than the database-only numbers in Table VII.

Within the evaluated wrapper reference path, the overhead comes from several implementation layers in the wrapper reference prototype. First, `taskbound.run` dispatches each query through a PL/pgSQL wrapper rather than through PostgreSQL's native planner and executor path alone. Second, each execution repeatedly resolves session claims and task state instead of caching a bound task descriptor in trusted backend state. Third, safe views add predicate and view-expansion cost even before SessionBound receipt or budget logic runs. Fourth, the wrapper path materializes result rows so that the runtime

TABLE IV
CURRENT ADVERSARIAL SQL EVALUATION SUMMARY FROM `adversarial_sql_20260719_162614.json`. DIRECT AGGREGATE AND WINDOW RELEASE IS DENIED PENDING APPROVED AGGREGATE TEMPLATES.

Bucket	Representative pattern	Result	Notes
Blocked direct violations	denied columns, raw schema, catalogs, GUC/session tampering, unsafe functions, prepared/cursor misuse, unsafe `COPY` variants, raw-table `COPY`, `EXPLAIN ANALYZE`, set operations, DDL/DML, replay/drift/rollback attempts	117 blocked / 117	direct attempts to leave the approved task boundary denied
Blocked payload aggregation	`json_agg`, `array_agg`, `string_agg`, XML/JSON builders, row/composite packing	6 blocked / 6	high-risk one-row payload compression denied by default
Blocked aggregate/window release	direct entity grouping, `HAVING` probes, filtered aggregate release, high configured k, ungrouped aggregate, and window release	7 blocked / 7	direct aggregate/window release requires future approved templates
Allowed safe-view detail queries	ordinary safe-view projection, join, bounded pagination, harmless aliasing, and non-aggregate CTE/detail access	10 allowed / 10	allowed answers were accounted and receipted
Total	all adversarial cases in `adversarial_sql_20260719_162614.json`	140 passed / 140	130 blocked, 10 allowed and accounted

TABLE V
CURRENT POSTGRESQL HOOK AND EXECUTOR ENFORCEMENT EVALUATION. AGENT CASES CONNECT DIRECTLY AS THE GENERATED RUNTIME CREDENTIAL AND ISSUE NATIVE SQL AFTER BINDING A SIGNED TASK TOKEN.

Case group	Enforcement path	Result	Notes
Trusted context	Non-superuser direct DB session	1 blocked / 1	Ordinary agent role cannot set SUSET guard GUCs
Native allowed surface	Agent credential direct DB session	4 allowed / 4	Bound `SELECT`, schema-qualified safe-view query, prepared `EXECUTE`, and `COPY (SELECT)` accounted
Native blocked surface	Agent credential direct DB session	8 blocked / 8	`UNION`, catalog access, recursive CTE, private helper access, cursor/`FETCH`, and `EXPLAIN` output denied
Hook-only structural checks	Trusted-GUC hook check without API preflight	6 blocked / 6	Non-`SELECT`, raw schema, payload aggregation, unapproved safe-view OID, direct entity `GROUP BY`, and `HAVING` denied

TABLE VI
SECURITY-PROPERTY COMPARISON ACROSS EVALUATED CONFIGURATIONS. THE RLS BASELINE INCLUDES ROW POLICY, FIELD-LIMITED VIEWS, SHORT CREDENTIALS, READ-ONLY GRANTS, AND AUDIT, BUT OMITS SESSIONBOUND TASK TOKENS, BUDGETS, DRIFT INVALIDATION, AND RECEIPT CHAINS.

Property	Raw	Role	Safe view	RLS+SV+Audit	SB full
Blocks writes	No	Yes	Yes	Yes	Yes
Blocks raw table access	No	No	Yes	Yes	Yes
Blocks denied fields	No	No	Yes	Yes	Yes
Enforces row scope	No	No	Yes	Yes	Yes
Query budget	No	No	No	No	Yes
Disclosure budget	No	No	No	No	Yes
Payload aggregation blocking	No	No	No	No	Yes
Credential-token binding	No	No	No	No	Yes
DB-local receipt hash chain	No	No	No	No	Yes
Basic audit log	No	No	No	Yes	Receipts
Schema drift token invalidation	No	No	No	No	Yes

can account for returned business identifiers. Fifth, receipt insertion and budget accounting add state updates, but the no-receipt and no-budget ablations show that these updates are not the dominant source on the small default dataset.

To separate structural enforcement from wrapper-era execution cost, we also ran a hook-only microbenchmark over the native `sessionbound_guard_check(sql)` path. This path prepares SQL through PostgreSQL parse/analyze and the structural guard, but does not execute result

TABLE VII
OVERHEAD P50 LATENCY IN MILLISECONDS FROM `overhead_breakdown_20260719_150951.json`. EACH SUPPORTED PATTERN/MODE USED 10 WARMUP AND 100 MEASURED ITERATIONS; "–" MEANS THE CURRENT SESSIONBOUND POLICY DENIES THAT RELEASE SHAPE.

Pattern	Raw	Role	Safe view	RLS+SV+Audit	SB no receipt	SB no budget	SB full
SELECT	0.236	0.267	0.431	1.280	153.960	152.948	152.923
JOIN	0.179	0.171	0.212	0.969	150.880	156.085	158.980
GROUP BY	0.225	0.206	0.260	1.110	–	–	–
CTE detail	0.204	0.180	0.214	0.910	155.381	157.473	151.161
Window	0.232	0.265	0.270	0.987	–	–	–

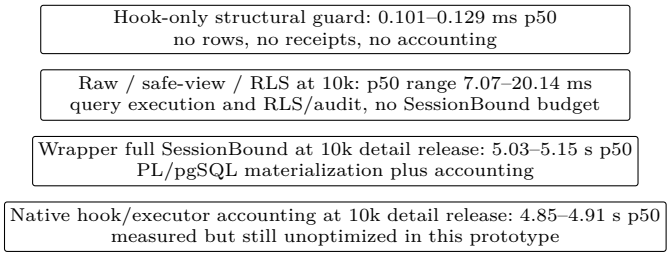

Fig. 3. Performance diagnosis. Structural guarding is sub-millisecond, while both accounting-complete paths become multi-second at 10k rows; the current bottleneck is result accounting/materialization rather than parse/analyze checks.

rows, perform executor row accounting, insert receipts, or materialize JSON rows through `taskbound.run`. With 20 warmup and 200 measured iterations per query shape, allowed structural checks had p50 latency of 0.101 ms for SELECT, 0.129 ms for JOIN, and 0.118 ms for non-aggregate CTE detail SQL; raw-schema, UNION, `GROUP BY`, and window denial checks also passed. This is not an end-to-end performance claim for the native executor-accounting path, but it shows that structural parse/analyze guarding is not the source of the 10k-row multi-second behavior.

The diagnostic native end-to-end benchmark in Table IX measures five baseline query shapes over 1k and 10k scoped rows using a bounded rerun with one warmup and three measured iterations per shape. For SessionBound, the supported shapes are detail `SELECT`, `JOIN`, and non-aggregate CTE detail queries; `GROUP BY` and window release are marked unsupported by policy. The SessionBound modes use API-issued runtime credentials and signed task tokens with elevated cumulative row budgets, but API calls are outside measured query latency. The wrapper mode executes through `taskbound.run(sql)`. The native mode binds the same task token and then issues direct safe-view SQL through the PostgreSQL hook and executor-accounting path.

The result is intentionally conservative. Safe-view-only and RLS+safe-view+audit baselines remain within roughly 20 ms p99 at 10k rows across these query shapes. Both current SessionBound accounting paths are much slower on 10k detail releases: wrapper p50 is 5.03–5.15 s and native hook/executor p50 is 4.85–4.91 s. Thus the hook-only structural result should not be presented as full SessionBound performance. The current native accounting path demonstrates direct DB enforcement semantics but has not yet delivered optimized scale behavior.

The 10k-row setting intentionally raises the disclosure budget to stress-test accounting completeness for larger detail releases. It should not be interpreted as the common operating mode for approved analytical tasks, where detail-release budgets are typically smaller and aggregate needs should be served by reviewed safe views or future approved aggregate templates. Enterprise audits may scan many rows without releasing every detail row to an agent; production deployments should optimize aggregate-template accounting and route high-cardinality detail exports through controlled workflows. The roadmap is cached task descriptors, executor accounting, `DestReceiver`/CustomScan-style integration, template-aware aggregate accounting, and less PL/pgSQL materialization. These optimizations target moving high-cardinality native accounting from PL/pgSQL-style materialization toward executor-local streaming overhead, with the engineering goal of reducing 10k-row detail-release accounting from multi-second latency to sub-second latency on similar hardware; this target remains future work rather than an evaluated claim.

G. Scale and concurrency

We separately tested credential-token and safe-view drift edge cases; the updated prototype denied all tested cases: credential A with token B, replaying a token with a second credential, same-session second active binding, wrong token audience or actor, expired token, revoked task state, registry and safe-view policy-version drift, view-definition hash mismatch, and exposed-column hash mismatch. The binding-lifecycle run also checks that the runtime snapshot carries database/view/dependency/option metadata and denies release after a view-option drift. Drift denials require re-approval before execution.

On the default seed (430 expenses, 240 employees, and 124 departments), an API concurrency smoke test completed 1, 5, and 20 concurrent sessions with zero failed sessions. At 20 concurrent sessions, query p50 was 82.896 ms and query p95 was 110.356 ms. This is an API-stack smoke test, not a throughput-capacity claim. The concurrent isolation script confirmed independent credentials, budgets, and task-specific receipts for two active tasks, and denied credential A with token B and same-backend double binding. In the concurrent isolation run, two active tasks executed interleaved queries with independent query-count, result-row, and conservative tuple counters; over-budget denial in one task did not change the other task's remaining budget or receipt chain.

TABLE VIII
GLOBAL SINGLE-ACTIVE BINDING EVALUATION FOR ONE EXACT (`TASK_ID`, `TOKEN_DIGEST`, `CREDENTIAL_ID`) KEY.

Case	Result
2 backends, same key	exactly 1 owner, 1 denial
10 backends, same key	exactly 1 owner, 9 denials
20 repeated races	0 dual-success, 0 zero-owner
Explicit unbind	immediate reacquisition
Socket close	stale recovery succeeds
Backend termination	stale recovery succeeds
Live idle owner	never time-evicted
Old fence mutation	rejected; 2 fenced events
Budget after recovery	preserved at 2 output tuples
Receipt chain after recovery	verified
Different binding keys	concurrent success

The global single-active binding evaluation specifically targets the same-key multi-backend gap. In `single_active_binding_20260719_065728.json`, two synchronized backends and a 10-contender race each produced exactly one owner. A 20-round synchronized race produced 20 single-winner rounds, zero dual-success rounds, zero zero-winner rounds, and zero unexpected errors. Explicit unbind, socket-close recovery, `pg_terminate_backend` recovery, rollback-independent binding, live-owner non-eviction, PID-reuse protection, stale fence rejection, lock/utility tamper blocking, budget continuity, receipt-chain continuity, and different-key concurrency all passed. Binding latency over the same run had p50 12.054 ms, p95 13.965 ms, and p99 15.992 ms.

We also ran a synthetic scale sweep over 1k and 10k scoped expense rows. Table IX reports p50 ranges across supported shapes, along with the maximum p95 and p99 observed across those patterns. Raw, safe-view, and RLS baselines include aggregate/window shapes; the SessionBound rows include only supported detail `SELECT`, `JOIN`, and non-aggregate CTE detail release, because aggregate/window release is denied by policy. All listed measurements completed with zero query errors. The 10k target is a detail-release stress setting with elevated budgets; these results expose a scale-sensitive limitation shared by the current wrapper and native accounting implementations and should not be read as a production-throughput claim.

H. Discussion of remaining gaps

Several gaps remain. The PostgreSQL hook path now includes native executor accounting, but the disclosure unit is still demo-specific and the native path is not optimized across broad production workloads. Direct ad-hoc aggregate and window release is denied pending approved aggregate templates, but arbitrary semantic inference across multiple allowed detail answers remains out of scope; the budget vector is operational and does not provide formal differential privacy.

XIV. RELATED WORK

SessionBound is not the first task-centered authorization model, contextual credential, database reference monitor, or agent tool-policy system. Its claim is narrower: it composes a signed enterprise task token, runtime database credential, registered safe-view surface, cumulative accounting, schema-drift checks, and receipt chain inside the SQL execution session.

TABLE IX
DIAGNOSTIC END-TO-END SCALE BENCHMARK IN MILLISECONDS FROM `NATIVE_END_TO_END_20260719_152051.JSON`. P50 IS THE RANGE ACROSS SUPPORTED QUERY SHAPES; P95/P99 ARE MAXIMA. THE RERUN USES ONE WARMUP PLUS THREE MEASURED ITERATIONS PER SHAPE AND ELEVATED DISCLOSURE BUDGETS AS A DETAIL-RELEASE STRESS TEST, NOT THE EXPECTED COMMON OPERATING MODE.

Rows	Mode	p50 range	max p95	max p99	Err.
1k	Raw	0.90–1.83	1.85	1.85	0
1k	Safe view	1.12–2.12	2.14	2.14	0
1k	RLS+SV+Audit	1.86–2.70	2.74	2.74	0
1k	SB wrapper	611.13–689.47	738.13	738.13	0
1k	SB native	618.96–650.39	687.16	687.16	0
10k	Raw	7.07–17.98	18.07	18.07	0
10k	Safe view	7.76–18.66	20.27	20.27	0
10k	RLS+SV+Audit	8.16–18.26	20.14	20.14	0
10k	SB wrapper	5030.59–5147.55	5511.51	5511.51	0
10k	SB native	4850.25–4906.16	5009.95	5009.95	0

TABLE X
NEAREST-NEIGHBOR POSITIONING. SB COMBINES MECHANISMS PRIOR WORK TREATS SEPARATELY; IT DOES NOT CLAIM A NEW ACCESS-CONTROL MODEL, FORMAL DP, OR COMPLETE SEMANTIC-INFERENCE PREVENTION.

Work	Unit	TCB	Cred. binding	SQL	State	Drift	Receipt/guarantee
TBAC/UCON [16], [17]	task / usage	model/app PEP	no DB login	no SQL RM	lifecycle	no view hashes	model semantics
Macaroons [18]	bearer caveat token	service verifier	caveat attenuation	no SQL mediation	context caveats	no DB drift	crypto delegation
Qapla [19]	user/time/row/column	DB adapter	app/user identity	SQL rewrite	query policy	no task snapshot	policy compliance
Blockaid [20]	web request/query	DB proxy + SMT	app/user identity	query verification	request/query	no task snapshot	query allow/block
Sieve [21]	querier/purpose/policy	middleware	app context	FGAC rewrite/UDF	query + policy set	no task snapshot	scalable FGAC
ShillDB [22]	component contract	language runtime	program capability	language DB ops	function contract	no DB drift	contract guarantee
Estrela [23]	API context/use	app framework	endpoint context	pre/post checks	request context	no DB snapshot	policy compliance
PICACHV [24]	data-use policy	RA monitor + TEE	analytics runtime	plan semantics	query plan	no task token	verified monitor
Agent policies [25], [26], [27]	task/tool/op	runtime/gateway	no DB login	tools/services	exec-state checks	no view drift	tool/operation auth
PAuth [3]	NL task operation	service verifier	envelopes/provenance	service calls	NL slices	no DB view drift	precise op auth
CaMeL/IFC [28], [29]	label / capability	interp./tool policy	labels/caps	data-flow mediation	taint/provenance	no DB drift	IFC guarantee
SessionBound	approved DB task	control plane/broker/PG	token + DB login	safe-view `SELECT`	budgets + binding	view/policy hashes	rollback receipts

ABAC/XACML, capability systems, purpose-aware access, OAuth, authenticated delegation, and Zanzibar provide authorization, delegation, or relationship checks [30], [31], [32], [33], [9], [10], [1], [2], [13]; SessionBound addresses how the resulting SQL session is scoped, budgeted, drift-checked, and audited.

PAuth authorizes service/tool operations implied by a natural-language task using NL slices and provenance envelopes [3]; Data Product MCP governs data-product discovery, access, and contract-aware querying [4], [5]. SessionBound is complementary: after approval or data-product selection, it enforces the PostgreSQL execution contract for open-ended SQL. Prompt-injection work, information-flow controls, RLS/VPD/DDS, DLP, parser-mediated validation, auditing, inference control, and differential privacy address adjacent pieces [34], [35], [36], [37], [29], [6], [7], [8], [38], [39], [40], [41], [42], [11], [14], [15]; database contract and data-use systems such as ShillDB, Estrela, and PICACHV enforce component-level database contracts, contextual pre/post-query policies, and verified data-use monitors, respectively. SessionBound composes task approval, credential binding, view-snapshot drift checks, cumulative disclosure accounting, and rollback-surviving receipts around one task-bound database session, not formal DP.

XV. Limitations

SessionBound is a research prototype and reference architecture, not a formal SQL safety proof. It targets one PostgreSQL database; production needs KMS/JWKS signing, credential lifecycle management, safe-view migration review, external audit retention, WORM anchoring, and external primary fencing for HA. The global active-binding protocol assumes one trusted writable PostgreSQL primary and is not a distributed consensus protocol; cross-cluster binding and database split brain are out of scope. Disclosure budgets are operational authority accounting, not differential privacy or complete semantic inference control: direct aggregate/window release is currently denied pending approved templates, but inference across allowed detail answers remains out of scope. Measurements cover microbenchmarks, ablations, concurrency smoke tests, and diagnostic 1k/10k scale benchmarks, not optimized broad production workloads. Federation, malicious DBAs, and compromised hosts are out of scope.

XVI. Conclusion

SessionBound turns approved tasks into budgeted database sessions for AI agents: "The agent can try. The database decides." It binds approval, credential identity, safe-view versions, SQL surface, budgets, and receipts into one database-enforced contract. Results show sub-millisecond structural guards and heavier 10k-row accounting paths; production work should optimize accounting and executor integration without weakening enforcement.

References

[1] D. Hardt, "The OAuth 2.0 Authorization Framework," RFC 6749, 2012. [Online]. Available: https://www.rfc-editor.org/info/rfc6749

[2] T. South, S. Marro, T. Hardjono, R. Mahari, C. D. Whitney, D. Greenwood, A. Chan, and A. Pentland, "Authenticated Delegation and Authorized AI Agents," 2025.

[3] R. K. Sharma, L. Jiang, Z. Lin, and S. Chen, "PAuth - Precise Task-Scoped Authorization For Agents," 2026.

[4] M. Tonnarelli, F. Scaramuzza, S. Harrer, and L. W. Dietz, "Data Product MCP: Chat with your Enterprise Data," 2026.

[5] Model Context Protocol, *Model Context Protocol Specification*, 2025. [Online]. Available: https://modelcontextprotocol.io/specification/2025-06-18

[6] PostgreSQL Global Development Group, *Row Security Policies*, PostgreSQL, 2026. [Online]. Available: https://www.postgresql.org/docs/current/ddl-rowsecurity.html

[7] Oracle, *Using Oracle Virtual Private Database to Control Data Access*, Oracle, 2026. [Online]. Available: https://docs.oracle.com/en/database/oracle/oracle-database/19/dbseg/using-oracle-vpd-to-control-data-access.html

[8] ——, *What Is Oracle Deep Data Security*, Oracle, 2026. [Online]. Available: https://docs.oracle.com/en/database/oracle/oracle-database/26/ddscg/what-is-oracle-deep-data-security.html

[9] R. Agrawal, J. Kiernan, R. Srikant, and Y. Xu, "Hippocratic Databases," in *Proceedings of the 28th International Conference on Very Large Data Bases*, 2002, pp. 143–154.

[10] J.-W. Byun and N. Li, "Purpose Based Access Control for Privacy Protection in Relational Database Systems," *The VLDB Journal*, vol. 17, no. 4, pp. 603–619, 2008.

[11] J. Kleinberg, C. Papadimitriou, and P. Raghavan, "Auditing Boolean Attributes," in *Proceedings of the Nineteenth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems*, 2000.

[12] S. U. Nabar, B. Marthi, K. Kenthapadi, N. Mishra, and R. Motwani, "Towards Robustness in Query Auditing," in *Proceedings of the 32nd International Conference on Very Large Data Bases*, 2006, pp. 151–162.

[13] R. Pang, R. Caceres, M. Burrows, Z. Chen, P. Dave, N. Germer, A. Golynski, K. Graney, N. Kang, L. Kissner, J. L. Korn, A. Parmar, C. D. Richards, and M. Wang, "Zanzibar: Google's Consistent, Global Authorization System," in *2019 USENIX Annual Technical Conference (USENIX ATC 19)*. USENIX Association, 2019, pp. 33–46.

[14] N. R. Adam and J. C. Wortmann, "Security-Control Methods for Statistical Databases: A Comparative Study," *ACM Computing Surveys*, vol. 21, no. 4, pp. 515–556, 1989.

[15] C. Dwork and A. Roth, *The Algorithmic Foundations of Differential Privacy*, ser. Foundations and Trends in Theoretical Computer Science. Now Publishers, 2014, vol. 9.

[16] R. K. Thomas and R. S. Sandhu, "Task-Based Authorization Controls (TBAC): A Family of Models for Active and Enterprise-Oriented Authorization Management," in *Database Security XI: Status and Prospects*. Springer, 1997.

[17] J. Park and R. S. Sandhu, "The UCONABC Usage Control Model,"

[18] A. Birgisson, J. G. Politz, Ú. Erlingsson, A. Taly, M. Vrable, and M. Lentczner, "Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud," in *Network and Distributed System Security Symposium*. Internet Society, 2014.

[19] A. Mehta, E. Elnikety, K. Harvey, D. Garg, and P. Druschel, "Qapla: Policy Compliance for Database-Backed Systems," in *26th USENIX Security Symposium (USENIX Security 17)*. USENIX Association, 2017, pp. 1463–1479.

[20] W. Zhang, E. Sheng, M. Chang, A. Panda, M. Sagiv, and S. Shenker, "Blockaid: Data Access Policy Enforcement for Web Applications," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. USENIX Association, 2022, pp. 701–718.

[21] P. Pappachan, R. Yus, S. Mehrotra, and J.-C. Freytag, "Sieve: A Middleware Approach to Scalable Access Control for Database Management Systems," *Proceedings of the VLDB Endowment*, vol. 13, no. 12, pp. 2424–2437, 2020.

[22] E. Zigmond, S. Chong, C. Dimoulas, and S. Moore, "Fine-Grained, Language-Based Access Control for Database-Backed Applications," *The Art, Science, and Engineering of Programming*, vol. 4, no. 2, p. 3, 2020.

[23] A. Bichhawat, M. Fredrikson, J. Yang, and A. Trehan, "Contextual and Granular Policy Enforcement in Database-backed Applications," in *Proceedings of the 15th ACM Asia Conference on Computer and Communications Security*. Association for Computing Machinery, 2020, pp. 479–491.

[24] H. H. Chen, H. Chen, M. Sun, C. Wang, and X. Wang, "PICACHV: Formally Verified Data Use Policy Enforcement for Secure Data Analytics," in *34th USENIX Security Symposium (USENIX Security 25)*. Seattle, WA: USENIX Association, 2025, pp. 5053–5070. [Online]. Available: https://www.usenix.org/conference/usenixsecurity25/presentation/chen-haobin

[25] T. Shi, J. He, Z. Wang, H. Li, L. Wu, W. Guo, and D. Song, "Progent: Programmable Privilege Control for LLM Agents," 2025.

[26] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents," 2025.

[27] F. Jia, T. Wu, X. Qin, and A. Squicciarini, "The Task Shield: Enforcing Task Alignment to Defend Against Indirect Prompt Injection in LLM Agents," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Vienna, Austria: Association for Computational Linguistics, 2025, pp. 29 680–29 697.

[28] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, "Defeating Prompt Injections by Design," 2025.

[29] M. Costa, B. Köpf, A. Kolluri, A. Paverd, M. Russinovich, A. Salem, S. Tople, L. Wutschitz, and S. Zanella-Béguelin, "Securing AI Agents with Information-Flow Control," 2025.

[30] V. C. Hu, D. Ferraiolo, R. Kuhn, A. Schnitzer, K. Sandlin, R. Miller, and K. Scarfone, "Guide to Attribute Based Access Control (ABAC) Definition and Considerations," National Institute of Standards and Technology, Special Publication 800-162, 2019.

[31] OASIS XACML Technical Committee, *eXtensible Access Control Markup Language (XACML) Version 3.0*, OASIS, 2013. [Online]. Available: https://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.html

[32] J. B. Dennis and E. C. Van Horn, "Programming Semantics for Multiprogrammed Computations," *Communications of the ACM*, vol. 9, no. 3, pp. 143–155, 1966.

[33] M. S. Miller, K.-P. Yee, and J. Shapiro, "Capability Myths Demolished," Johns Hopkins University Systems Research Laboratory, Tech. Rep. SRL2003-02, 2003.

[34] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, 2023.

[35] Y. Liu, G. Deng, Y. Li, K. Wang, T. Zhang, Y. Liu, H. Wang, Y. Zheng, and Y. Liu, "Prompt Injection Attack against LLM-integrated Applications," 2023.

[36] OWASP GenAI Security Project, *LLM01:2025 Prompt Injection*, OWASP, 2025. [Online]. Available: https://genai.owasp.org/llmrisk/llm01-prompt-injection/

[37] A. C. Myers and B. Liskov, "A Decentralized Model for Information Flow Control," in *Proceedings of the Sixteenth ACM Symposium on Operating Systems Principles*, 1997, pp. 129–142.

[38] Oracle, *Oracle Deep Data Security Technical Brief*, Oracle, 2026. [Online]. Available: https://www.oracle.com/a/ocom/docs/security/deep-data-security-technical-brief.pdf

[39] S. Alneyadi, E. Sithirasenan, and V. Muthukkumarasamy, "A Survey on Data Leakage Prevention Systems," *Journal of Network and Computer Applications*, vol. 62, pp. 137–152, 2016.

[40] National Institute of Standards and Technology, *Data Loss Prevention*, NIST Computer Security Resource Center Glossary, 2026. [Online]. Available: https://csrc.nist.gov/glossary/term/data_loss_prevention

[41] S. W. Boyd and A. D. Keromytis, "SQLrand: Preventing SQL Injection Attacks," in *Applied Cryptography and Network Security*. Springer, 2004, pp. 292–302.

[42] Z. Su and G. Wassermann, "The Essence of Command Injection Attacks in Web Applications," in *Proceedings of the 33rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 2006, pp. 372–382.